



ĐẠI HỌC ĐÀ NẴNG
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG VIỆT - HÀN
Vietnam - Korea University of Information and Communication Technology

JAVA NÂNG CAO (Advanced Java)

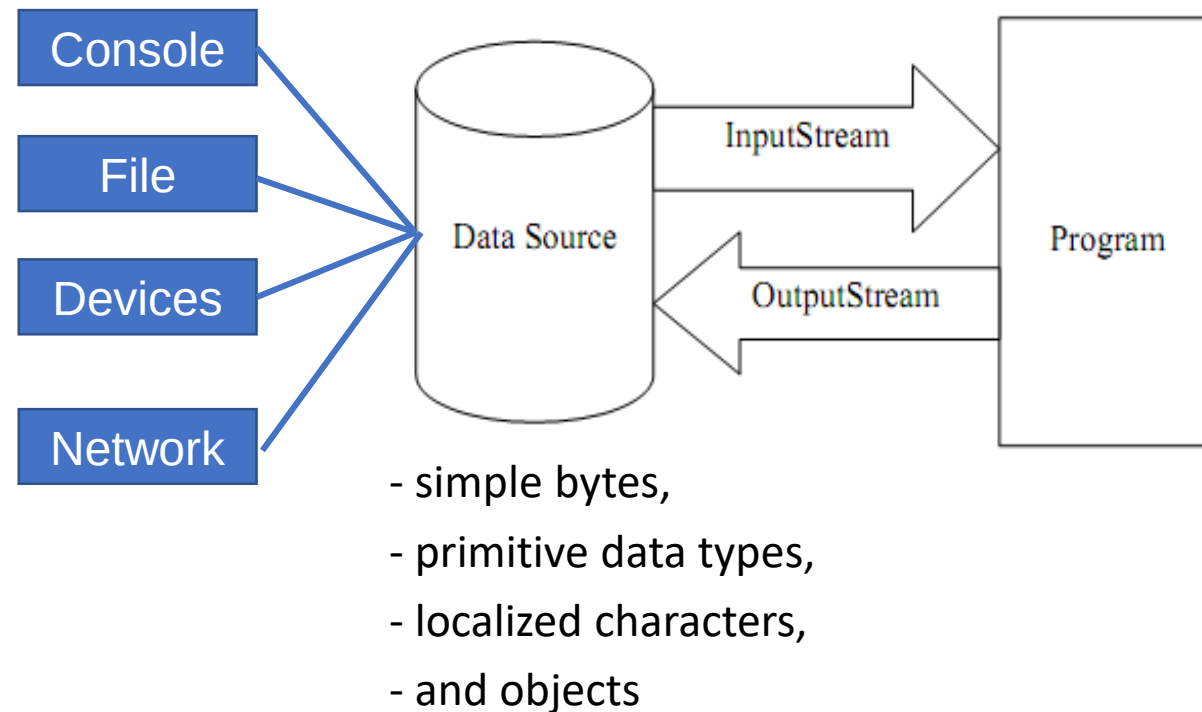
IO Stream



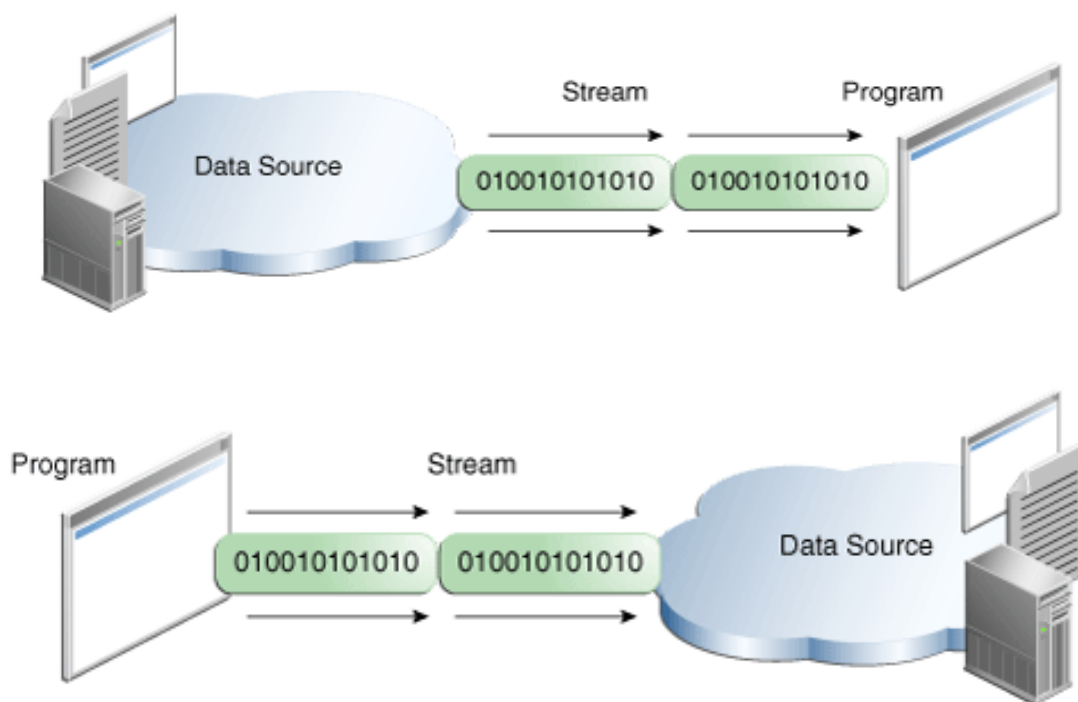
CONTENTS

1. IO Streams
2. File Streams
3. Filter Streams
4. Buffered Streams
5. Reader class và Writer class
6. CharArrayReader class and CharArrayWriter class
7. StringReader class and StringWriter class

- Java I/O is based on the notation of streams.
- Streams are sequences of data.
- Advantages of streams:
 - No need to learn inner working of each device separately.
 - The program will work for different input and output devices without any code changes.

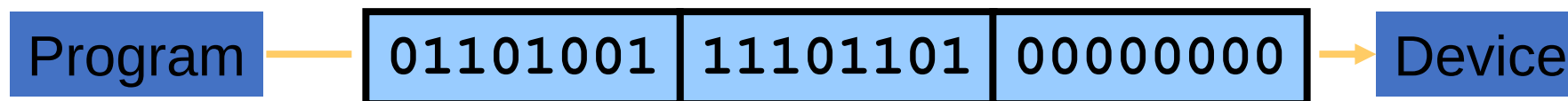


- In a program, we read information from an input stream and write information to an output stream.



- A stream can represent many different kinds of sources and destinations:
 - Disk files
 - Devices
 - Programs
 - Network socket
 - Memory arrays
 - ...
- Streams support many different kinds of data:
 - Simple bytes
 - Primitive data types
 - Localized characters
 - Objects
- A program can manage multiple streams at a time.

- The stream classes are divided into two types, based on the data type:
 - Byte Stream:
 - For binary data
 - Containing 8 - bit information
 - Character Stream:
 - For Unicode characters



- The java.io package contains a collection of stream classes that supports for reading and writing.
- Classes for byte streams:
 - The **InputStream** Class
 - The **OutputStream** Class
 - Both classes are **abstract**
- Classes for character streams:
 - The **Reader** class
 - The **Writer** class
 - Both classes are **abstract**

- o class java.lang.[Object](#)
 - o class java.io.[InputStream](#)
 - o class java.io.[ByteArrayInputStream](#)
 - o class java.io.[FileInputStream](#)
 - o class java.io.[FilterInputStream](#)
 - o class java.io.[OutputStream](#)
 - o class java.io.[ByteArrayOutputStream](#)
 - o class java.io.[FileOutputStream](#)
 - o class java.io.[FilterOutputStream](#)
 - o class java.io.[Reader](#)
 - o class java.io.[BufferedReader](#)
 - o ...
 - o class java.io.[InputStreamReader](#)
 - o class java.io.[Writer](#)
 - o class java.io.[BufferedWriter](#)
 - o ...
 - o class java.io.[OutputStreamWriter](#)

▪ **Exception:**

- while reading/writing in stream, errors may be occurs.
- **IOException** is thrown.

Example:

```
public static void main(String args[]) throws IOException
```

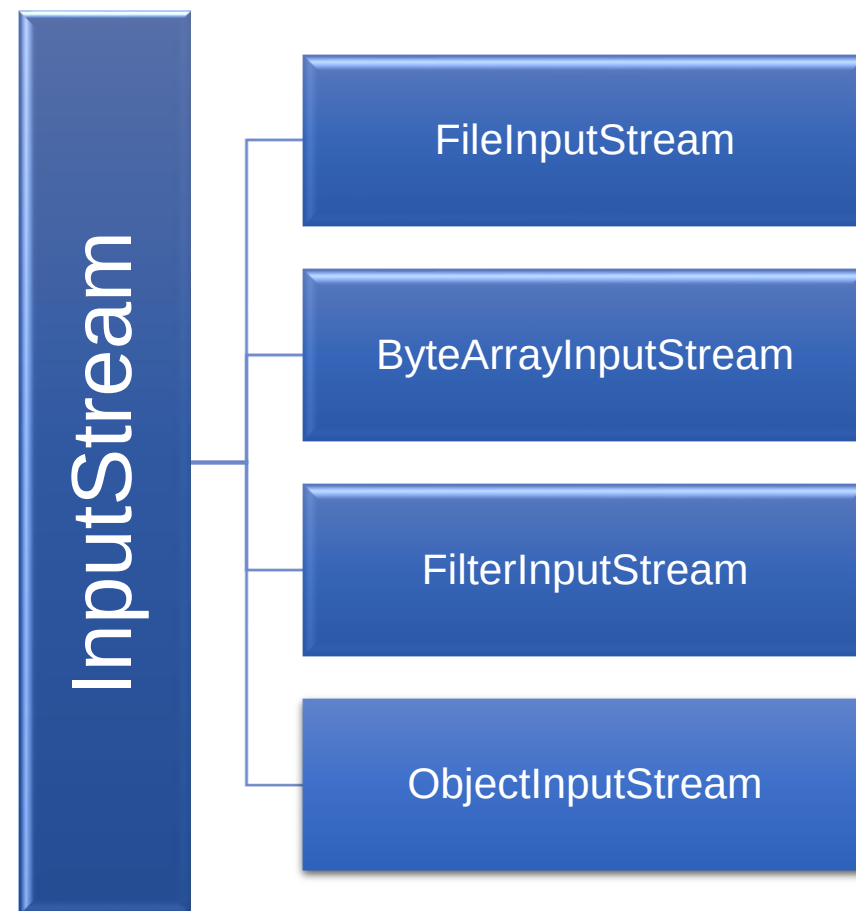
Example: Using try/ catch block:

```
try {  
    copy(fi, fo);  
} catch (IOException ex) {  
    System.err.println(ex);  
}
```

- To read and write 8-bit bytes, programs should use the byte streams, descendants of `InputStream` and `OutputStream`.
- `InputStream` and `OutputStream` provide the API and partial implementation for input streams (streams that read 8-bit bytes) and output streams (streams that write 8-bit bytes).

InputStream class

- The basic purpose: read data from an input stream.
- Many subclasses:
 - FileInputStream
 - ByteArrayInputStream
 - FilterInputStream
 - ObjectInputStream





Methods in InputStream class

- **read:** Read every byte from file (in byte):

- **int read()**

Return value (8 low bits of int): 0-255; end of stream: -1

- **int read(byte[] b)**

Read data from input stream to array b

- **int read(byte[] b, int offs, int len)**

Read data from input stream to array b, start offs, save len bytes

- **public int available()**

- Numbers data bytes left in Stream

- **public long skip(long count)**

- Skip count data bytes

- **public void reset()**

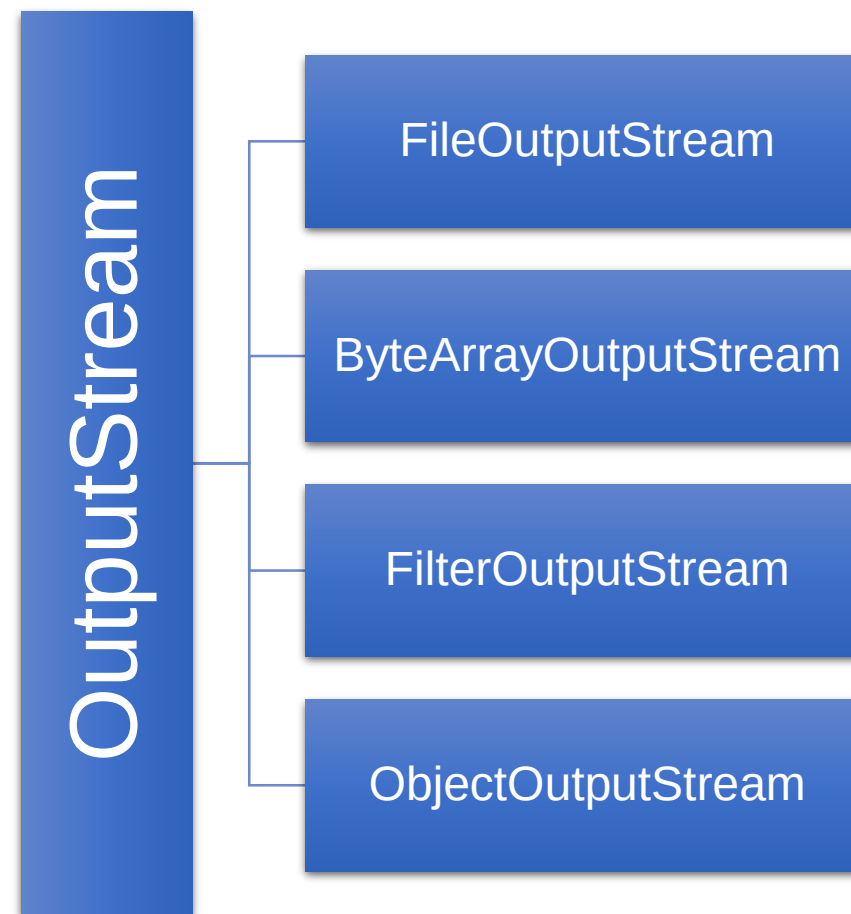
- Redefine recent marked position in InputStream .

- **public void close()**

- Close file. Release resource

OutputStream class

- The basic purpose: write data to a stream.
- Many subclasses:
 - FileOutputStream
 - ByteArrayOutputStream
 - FilterOutputStream
 - ObjectOutputStream





Methods in OutputStream class

▪ **write:** Write every byte to file:

- **void write(int b)**

Write every byte to file (8 low bits of int).

$b = 0-255$; others, $b = b \bmod 256$

- **void write(byte[] b)**

Write bytes from output stream to array b

- **void write(byte[] b, int offs, int len)**

Write bytes from output stream to array b, start `offs`, save `len` bytes

▪ **flush()**

- Close file

ByteArrayInputStream class

- Is an implementation of an input stream that uses the byte array as the source.
- There are two constructors:
 - **ByteArrayInputStream(byte array[])**
 - creates a `ByteArrayInputStream` with **array** as the input source.
 - **ByteArrayInputStream(byte array[], int start, int num)**
 - creates a `ByteArrayInputStream` that begins with the character at `start` position and is `num` bytes long.
- Methods: **read()** , **skip()** , **reset()** , **available()**



ByteArrayOutputStream class

- Is an implementation of an output stream that uses the byte array as the destination.
- There are two constructors:
 - **ByteArrayOutputStream()**
 - used to set the output byte array to an initial size.
 - **ByteArrayOutputStream(int numBytes)**
 - sets the output buffer to a default size.
- Additional methods like:
 - **String toByteArray()**: convert the stream to a byte array object respectively
 - **String toString()**: convert the stream to a String object respectively
 - **size()**: size of buffer



Example: ByteArrayOutputStream class

```
public class ByteArrayIOApp {
    public static void main(String args[]) throws IOException {
        ByteArrayOutputStream outputStream = new ByteArrayOutputStream();
        String s = "This is a test.";
        for (int i = 0; i < s.length(); ++i)
            outputStream.write(s.charAt(i));
        System.out.println("outstream: " + outputStream);
        System.out.println("size: " + outputStream.size());
        ByteArrayInputStream inputStream;
        inputStream = new ByteArrayInputStream(outputStream.toByteArray());
        int inBytes = inputStream.available();
        System.out.println("inputStream has" + inBytes + "available bytes");
        byte inBuf[] = new byte[inBytes];
        int bytesRead = inputStream.read(inBuf, 0, inBytes);
        System.out.println(bytesRead + " bytes were read");
        System.out.println("They are: " + new String(inBuf));
    }
}
```

- Java File class represents the files and directory pathnames in an abstract manner. This class is used for creation of files and directories, file searching, file deletion, etc.
- The File object represents the actual file/directory on the disk. Following is the list of constructors to create a File object.
- Constructors:
 - File(File parent, String child)
 - File(String pathname)
 - File(String parent, String child)
 - File(URI uri)



Example: File class

```
import java.io.File;
public class FileDemo {
    public static void main(String[] args) {
        File f = null;
        String[] strs = {"test1.txt", "test2.txt"};
        try {
            // for each string in string array
            for(String s:strs ) {
                f = new File(s); // create new file
                boolean bool = f.canExecute(); // true if the file is executable
                String a = f.getAbsolutePath(); // find the absolute path
                System.out.print(a); // prints absolute path
                System.out.println(" is executable: "+ bool); // prints
            }
        } catch (Exception e) {
            e.printStackTrace(); // if any I/O error occurs
        }
    }
}
```

FileDescriptor class

- FileDescriptor class serves as an handle to the underlying machine-specific structure representing an open file, an open socket, or another source or sink of bytes. The handle can be err, in or out.
- The FileDescriptor class is used to create a FileInputStream or FileOutputStream to contain it.
- Constructors:
 - `FileDescriptor()`

FileInputStream class

- This creates an `InputStream` that we can use to read bytes from a file.
- Constructors of this class :
 - **`FileInputStream(String filename)`** : Creates an `InputStream` that we can use to read bytes from a `filename` (string).
 - **`FileInputStream(File filename)`** : Creates an input stream that we can use to read bytes from a file where `filename` is a `File` object.
- Exception: **`FileNotFoundException`**

FileOutputStream class

- This creates an OutputStream that we can use to Write bytes to a file.
- Four constructors are there, Its constructors are:
 - **FileOutputStream(String filename):**
Creates an OutputStream, to write bytes to a file.
 - **FileOutputStream(File name):**
Creates an OutputStream, to write bytes to a file.
 - **FileOutputStream(String filename, boolean flag) :**
Creates an OutputStream, to write bytes to a file. If flag is true, file is opened in append mode.
- Exception: **FileNotFoundException**

Example: read from file

```
import java.io.*;
class FileDemo {
    public static void main(String args[]) throws Exception {
        InputStream f = new FileInputStream("a.txt");
        int size= f.available();
        System.out.println("Bytes available: " + size);
        byte[] buff = new byte[size];
        f.read(buff,0,size);
        String s = new String(buff);
        System.out.print(s);
        f.close();
    }
}
```

```
while (f.available() > 0) {
    char ch = (char) f.read();
    System.out.print(ch);
}
```



Example: write data (bytes) to file

```
import java.io.*;
class FileOutputDemo {
    public static void main(String args[]) {
        String s="This is a test.";
        byte b[] = s.getBytes();
        try {
            FileOutputStream fos = new FileOutputStream("aa.txt");
            fos.write(b,0,b.length);
            System.out.println(b.length + " bytes are written!");
        } catch(IOException e) {
            System.out.println("Error creating file!");
        }
    }
}
```



Example: Copy a file

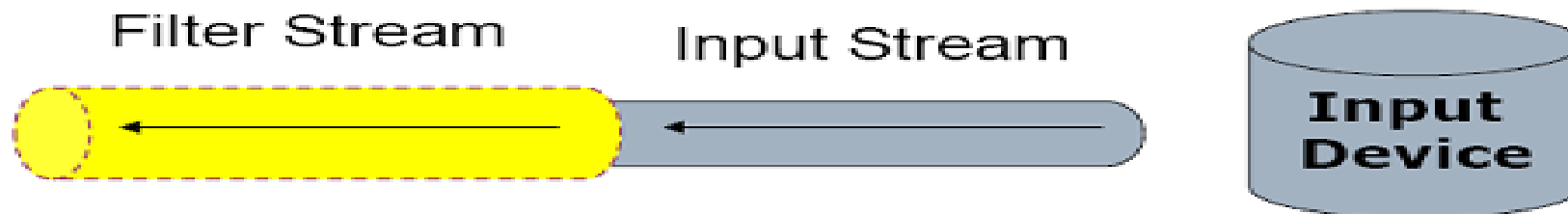
```
public class CopyFileTV_ex {  
    public static void main(String[] args) throws IOException {  
        FileInputStream fin = null;  
        FileOutputStream fout = null;  
        fin = new FileInputStream("a1.txt");  
        fout = new FileOutputStream("xyz.txt");  
        byte[] buffer = new byte[1024];  
        int bytesRead;  
        while ((bytesRead = fin.read(buffer)) > 0) {  
            fout.write(buffer, 0, bytesRead);  
        }  
        fin.close();  
        fout.close();  
    }  
}
```



Example: FileDescriptor

```
import java.io.*;
public class FileDescriptorExample {
    public static void main(String[] args) {
        FileDescriptor fd = null;
        byte[] b = { 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58 };
        try {
            FileOutputStream fos = new FileOutputStream("Record.txt");
            FileInputStream fis = new FileInputStream("Record.txt");
            fd = fos.getFD();
            fos.write(b);
            fos.flush();
            fd.sync(); // confirms data to be written to the disk
            int value = 0;
            while ((value = fis.read()) != -1) { // for every available bytes
                char c = (char) value; // converts bytes to char
                System.out.print(c);
            }
            System.out.println("\nSync() successfully executed!!");
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

- The java.io package provides a set of abstract classes that define and partially implement **filter streams**.
- A filter stream filters data as it's being read from or written to the stream.
- The filter streams are:
 - FilterInputStream
 - FilterOutputStream



Constructors of Filter Streams

- **BufferedOutputStream**

- `BufferedInputStream(InputStream is)`: Creates a buffered input stream for the specified `InputStream` instance.
- `BufferedInputStream(InputStream is, int size)`: Creates a buffered input stream of a given size for the specified `InputStream` instance.

- **BufferedOutputStream**

- `BufferedOutputStream(OutputStream os)`: Creates a buffered output stream for the specified `OutputStream` instance with a buffer size of 512.
- `BufferedOutputStream(OutputStream os, int size)`: Creates a buffered output stream of a given size for the specified `OutputStream` instance.



Example: Read from file with BufferedInputStream

```
public class BuffInput {
    public static void main(String[] args) throws Exception {
        byte[] buffer = new byte[1024];
        BufferedInputStream bi=new BufferedInputStream(new FileInputStream("b.txt"));
        int bytesRead = 0;
        while ((bytesRead = bi.read(buffer)) != -1)
        {
            String chunk = new String(buffer, 0, bytesRead);
            System.out.print(chunk);
        }
        bi.close();
    }
}
```

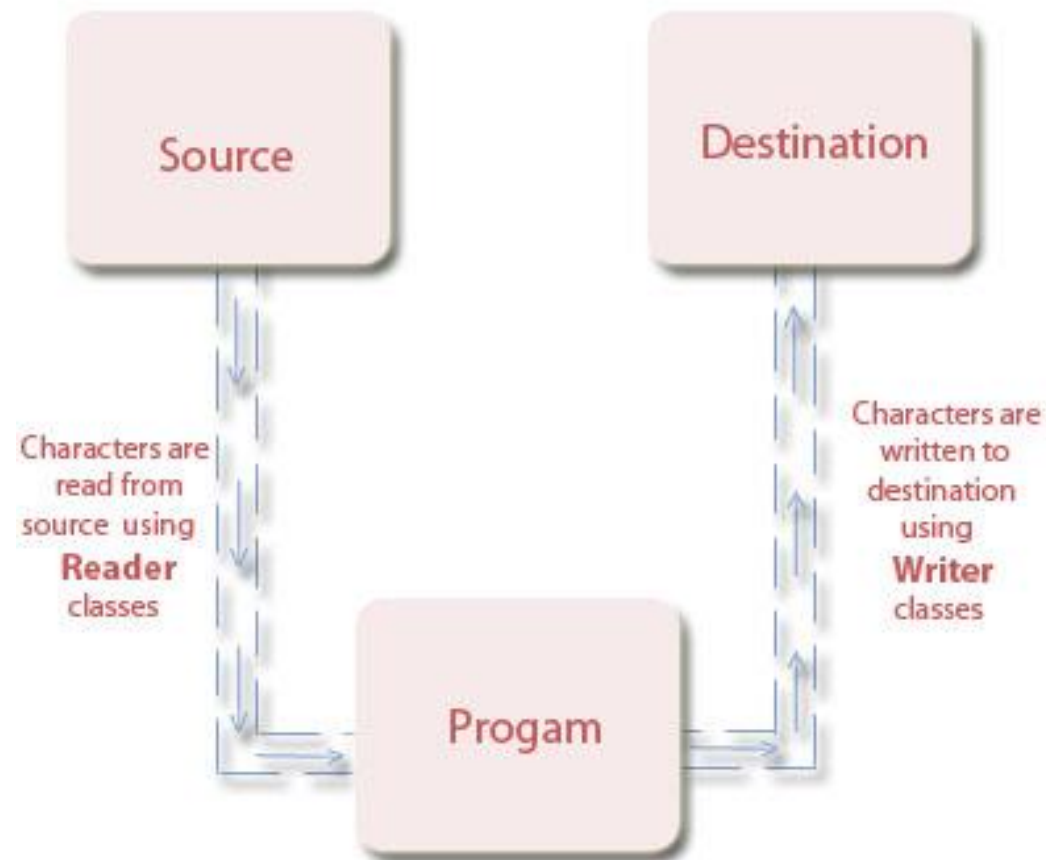


Example: Buffered Stream Copier

```
public class BuffOutput {
    public static void main(String[] args) throws IOException {
        FileInputStream fi= new FileInputStream("a.txt");
        FileOutputStream fo= new FileOutputStream("b.txt");
        try {
            copy(fi, fo);
        } catch (IOException ex) { System.err.println(ex); }
    }
    public static void copy(FileInputStream in, FileOutputStream out) throws IOException{
        BufferedInputStream bin = new BufferedInputStream(in);
        BufferedOutputStream bout = new BufferedOutputStream(out);
        while (true) {
            int datum = bin.read();
            if (datum == -1) break;
            bout.write(datum);
        }
        bout.flush();
    }
}
```

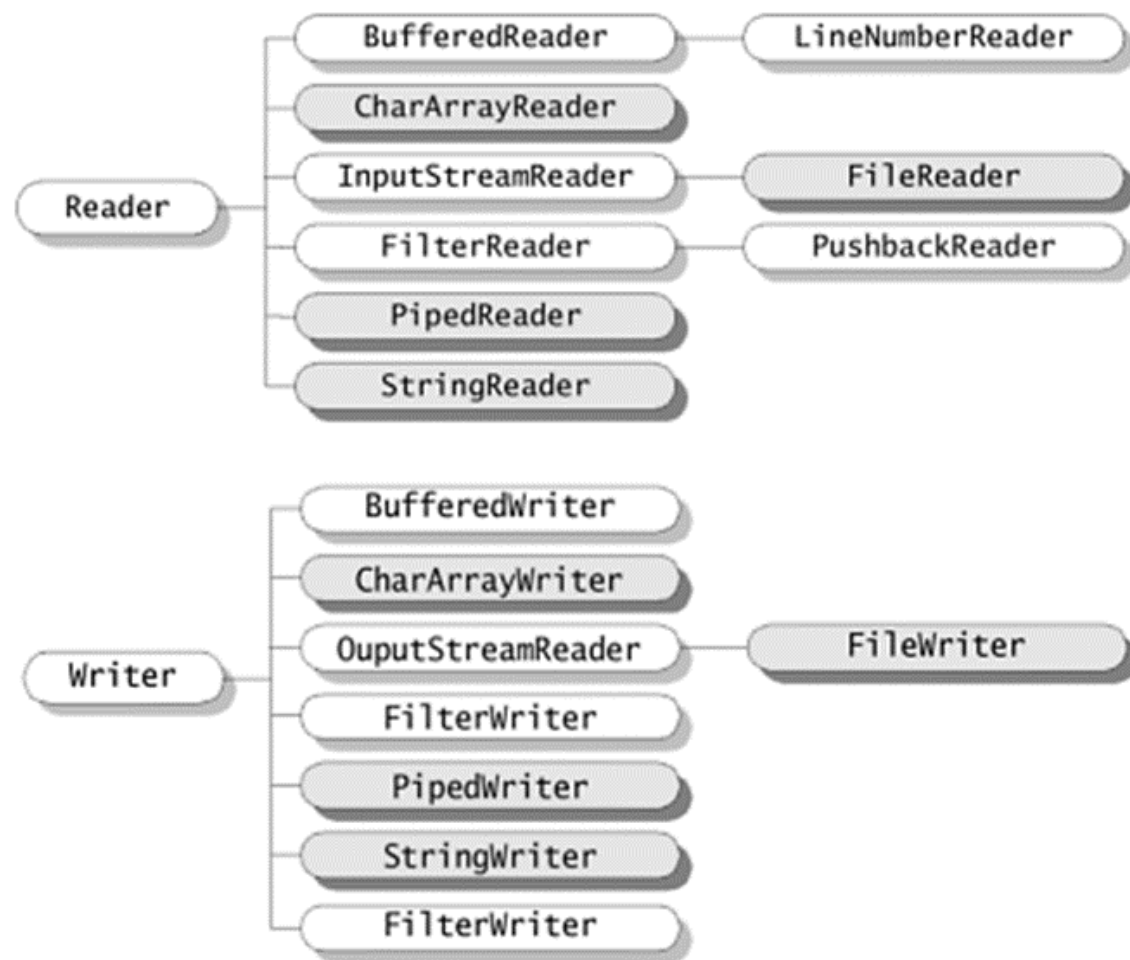
Character Streams

- Character streams are used for storing and retrieving text.
- All binary numeric data has to be converted to a textual representation before being written to a character stream.
- Stream Readers and Writers are objects that can read and write byte streams as character streams.



The class hierarchies for the Reader and Writer classes

- Reader and Writer are the abstract super classes for character streams in java.io.
- Reader provides the API and partial implementation for readers (streams that read 16-bit characters).
- Writer provides the API and partial implementation for writers (streams that write 16-bit characters).



- **abstract void close()**: Closes the Input source and after closing if we attempt to read it will give IOException.
- **void mark()**: Places the mark at the current point in the input stream.
- **int read()**: Returns an integer representing of the next available character of input. Returns -1 if EOF encountered.
- **void reset()**: Resets the input pointer to the previously set mark.
- **boolean ready()**: Returns True if the next input request will not wait. Otherwise, it returns False.

Methods in Writer Class

- **abstract void close()**: Closes the output stream.
- **abstract void flush()**: Finalizes the output state so that any buffers are cleared. That is it flushes the output buffers.
- **void write()**: Writes a single character to the invoking stream.
- **void write(char buffer[])**: Writes a complete array of character to the invoking stream.
- **void write(str)**: Writes a str to the invoking stream.



FileReader class and FileWriter class

■ **FileReader**

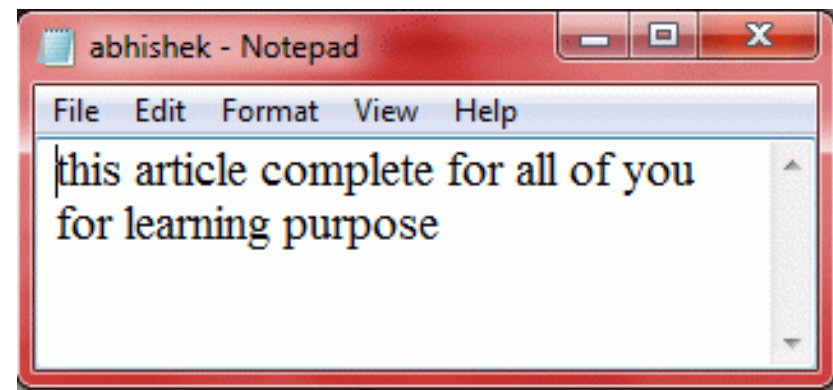
- This class creates a Reader that you can use to read the contents of the file.
- The constructors in this class:
 - `FileReader(String filepath)`
 - `FileReader(File fileobj)`

■ **FileWriter**

- This class creates a Writer that you can use to write the contents of the file.
- The constructors in this class:
 - `FileWriter(String filepath)`
 - `FileWriter(File fileobj)`

Example: FileWriter class

```
import java.io.*;
class MyFileWriter {
    public static void main(String arg[]) throws IOException {
        //first we make the object of FileWriter class
        FileWriter fw = new FileWriter("abhishek.txt");
        //this is a string and using write in "abhishek.txt" file
        String s = "this article complete for all of you for learning purpose";
        //toCharArray() is method to convert a string in a character array
        char ch[] = s.toCharArray();
        for (int i = 0; i < ch.length; i++)
            fw.write(ch[i]);
        fw.close();
    }
}
```





Example: FileReader class

```
import java.io.*;
class MyFileReader {
    public static void main(String arg[]) throws IOException {
        int i = 0;
        FileReader fr = new FileReader("abhishek.txt");
        while ((i = fr.read()) != -1)
            System.out.print((char) i);
        fr.close();
    }
}
```



BufferedReader class and BufferedWriter class

■ **BufferedReader**

- Streams are wrapped to combine the various features of the many streams.
- The constructors in this class are
 - `BufferedReader(Reader inputStream)`
 - `BufferedReader(Reader inputStream, int bufSize)`

■ **BufferedWriter**

- The writer that adds the `flush()` method that can be used to ensure that data buffers are physically written to the actual output stream
- Using `BufferWriter` we can increase performance by reducing the number of times data is actually physically written to the output stream.
- The constructors in this class are
 - `BufferedWriter(Writer outputStream)`
 - `BufferedWriter(Writer outputStream, int bufSize)`



Example: BufferedReader class

```
import java.io.*;
public class ReadFile {
    public static void main(String[] args) {
        try {
            File file = new File("file.txt");
            FileReader fileReader = new FileReader(file);
            BufferedReader bufferedReader = new BufferedReader(fileReader);
            String line;
            while((line = bufferedReader.readLine()) != null) {
                System.out.println(line);
            }
            bufferedReader.close();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```



Example: BufferedWriter class

```
import java.io.*;
public class WriteFile {
    public static void main(String[] args) {
        String[] list = {"one", "two", "three", "four"};
        try {
            File file = new File("file.txt");
            FileWriter fileReader = new FileWriter(file);
            BufferedWriter bufferedWriter = new BufferedWriter(fileReader);
            for (String s : list) {
                bufferedWriter.write(s + "\n");
            }
            bufferedWriter.close();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```



Example: We read data from the console and write it to a file

```
import java.io.*;
class ConsoleRead {
    public static void main(String[] args) {
        try {
            File file = new File("file.txt");
            InputStreamReader inputStreamReader = new InputStreamReader(System.in);
            BufferedReader bufferedReader = new BufferedReader(inputStreamReader);
            FileWriter fileWriter = new FileWriter(file);
            BufferedWriter bufferedWriter = new BufferedWriter(fileWriter);

            String line;
            while(!(line = bufferedReader.readLine()).equals("exit")) {
                bufferedWriter.write(line);
            }
            bufferedReader.close();
            bufferedWriter.close();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```



CharArrayReader class and CharArrayWriter class

■ CharArrayReader

- This class is an implementation of an input stream that uses the char array as source.
- This class has two constructors, each of which requires a character array to provide the data source:
 - `CharArrayReader(char array[ ])`
 - `CharArrayReader(char array[ ], int start, int numChars)`

■ CharArrayWriter

- This class is an implementation of an output stream that uses the char array as destination.
- This class has two constructors:
 - `CharArrayWriter(Char array[])`
 - `CharArrayWriter(int numChars)`



Example: CharArrayReader class

```
import java.io.CharArrayReader;
import java.io.IOException;
public class CharArrayReaderExample {
    public static void main(String[] args) {
        char ch[] = "This is an example of CharArrayReader.".toCharArray();
        CharArrayReader charArrayReader = null;
        try {
            charArrayReader = new CharArrayReader(ch);
            // Read characters
            int c;
            while ((c = charArrayReader.read()) != -1) {
                System.out.print((char) c);
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```



Example: CharArrayWriter class

```
import java.io.CharArrayWriter;
import java.io.IOException;
public class CharArrayWriterExample {
    public static void main(String[] args) {
        CharArrayWriter charArrayWriter = new CharArrayWriter();
        try{
            // Write characters to Writer
            charArrayWriter.write("This is an example of CharArrayWriter");
            // Get character array from writer
            char[] ch = charArrayWriter.toCharArray();
            for (char c : ch) {
                System.out.print(c);
            }
        } catch(IOException e) {
            e.printStackTrace();
        }
    }
}
```



StringReader class and StringWriter class

- A `StringReader` class converts the ordinary string into reader, while a `StringWriter` class collects the character in a string buffer, which is used to construct a string.
- **`Java.io.StringReader` class** is a character stream with string as a source. It takes an input string and changes it into character stream. It inherits `Reader` class. In **`StringReader` class**, system resources like network sockets and files are not used, therefore closing the **`StringReader`** is not necessary.
- **`java.io.StringWriter` class** creates string from the characters of the `String` Buffer stream. Methods of the `StringWriter` class can also be called after closing the closing the Stream as this will raise no `IO Exception`.



Example: StringReader class

```
import java.io.IOException;
import java.io.StringReader;

public class StringReaderExample {
    public static void main(String[] args) {
        String input = "This is an example of StringReader.";
        StringReader stringReader = new StringReader(input);
        int c;
        try {
            while ((c = stringReader.read()) != -1) {
                System.out.print((char) c);
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```



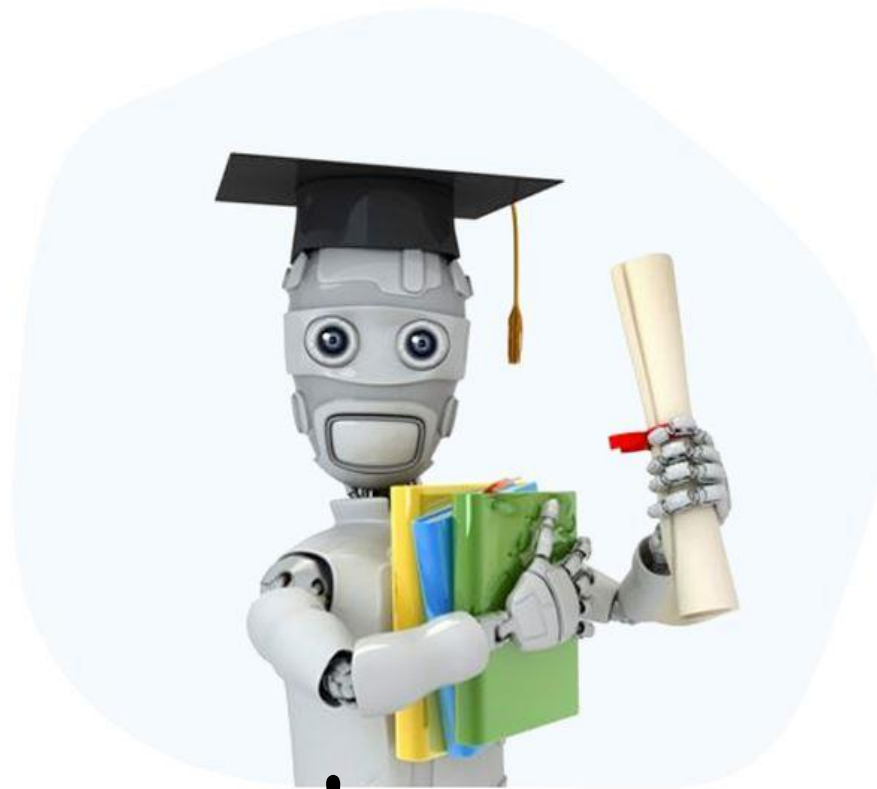
Example: StringWriter class

```
import java.io.StringWriter;

public class StringWriterExample {
    public static void main(String[] args) {
        StringWriter stringWriter = new StringWriter();
        stringWriter.write("This is an example ");
        stringWriter.write("of StringWriter.");

        // Convert writer to String
        System.out.println(stringWriter.toString());
    }
}
```





Thank you...!